



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Sistemas e Industrial
Curso: Ingeniería de Software 1 (2016701)
Grupo Kernel Panic

integrantes:

Pablo Rosales Obando

Gabriel Bustos Moreno

Alexander Mosquera Ramirez

Diego Esteban Jimenez Gutierrez

1. Contexto del proyecto

El sistema "Mentora" está diseñado para resolver la desorganización, el ausentismo y la falta de seguimiento efectivo en los procesos de tutorías académicas institucionales. Actualmente, la gestión manual o fragmentada de estos espacios dificulta que los estudiantes encuentren horarios, que los tutores planifiquen sesiones basándose en el historial del alumno, y que las instituciones midan la demanda real.

A diferencia de calendarios genéricos, "Mentora" integra un ciclo académico completo: desde la asignación de cuestionarios previos, pasando por un agendamiento inteligente, hasta el registro de calificaciones y observaciones, unificando todo en un flujo de desempeño medible. La población objetivo abarca a estudiantes, tutores, profesores y administradores de la institución educativa.

2. Roles y usuarios del sistema

2.1 Roles del equipo de desarrollo

- Alexander Mosquera Ramírez (Arquitecto de software): Será responsable de definir la arquitectura del sistema y tomar decisiones técnicas clave sobre la estructura y tecnologías. Diseñará una base sólida y escalable.
- Pablo Rosales Obando (Desarrollador backend): Se encargará de implementar la lógica de negocio, la conexión con la base de datos y el manejo de la información.
- Diego Esteban Jimenes Gutierrez (Desarrollador frontend): Se encargará de diseñar e implementar la interfaz gráfica del usuario. Asegurará que la plataforma sea intuitiva, accesible y cumpla con criterios de diseño responsivo para garantizar una navegación fluida en dispositivos móviles y de escritorio.
- Gabriel Bustos Moreno (Analista funcional): Gestionará la comunicación entre el equipo técnico y los usuarios, asegurando que los requerimientos estén bien documentados y el sistema cumpla las expectativas.

2.2 Usuarios finales del sistema

- **Estudiante:** Podrá agendar, cancelar o reprogramar tutorías. Además, consultará su historial académico, responderá cuestionarios y revisará las observaciones de sus tutores.
- **Tutor:** Gestionará su disponibilidad horaria, consultará el historial de los alumnos asignados y registrará calificaciones/observaciones para el seguimiento académico.

- **Profesor:** Evaluará el nivel de los alumnos creando y asignando cuestionarios previos a las tutorías.
- **Administrador:** Gestionará las cuentas de todos los usuarios, analizará reportes de desempeño institucional y supervisará la demanda del sistema

3. Listado de requisitos

ID	MoSCoW	DESCRIPCIÓN	TAREAS	DÍAS
RF_01	MUST	Como usuario: Quiero un sistema de login que me permita acceder según mi rol de estudiante tutor o profesor para visualizar las herramientas específicas de mi cargo.	Base de datos	3
			Lógica Back	3
			Interfaz con usuario front	3
RF_02	MUST	Como profesor: Quiero crear y asignar cuestionarios específicos a las materias para que Los estudiantes respondan	Lógica para crear cuestionario	3
			Visualización del cuestionario front	3
RF_03	COULD	Como sistema: Quiero enviar notificaciones automáticas 24 horas antes de la tutoría para reducir el ausentismo(por la app o email)	Recordatoria por app	3
			Recordatorio por email	3
RNF_04	WON'T	Como desarrollador quiero que la aplicación sea desarrollada bajo una arquitectura monolítica utilizando una base de datos relacional.	Crear la base de datos	3
RF_05	MUST	Como estudiantes quiero agendar una tutoría seleccionando fecha, hora y materia para asegurar mi cupo con el tutor.	BD: Modelo de disponibilidad, cupos y relación tutor-materia	2
			Backend: Lógica de	3

			agendamiento, validación de conflictos y asignación de cupo	
			Frontend: Formulario de selección de fecha, hora y materia	3
			Integración y pruebas	1
RF_06	Should	Como tutor, quiero registrar observaciones y calificaciones después de cada tutoría para hacer seguimiento al desempeño académico del estudiante	BD: Modelo de observaciones y calificaciones por sesión	2
			Backend: Endpoints para crear y editar registros post-tutoría	3
			Frontend: Formulario de registro accesible desde el perfil del estudiante	3
			Integración y pruebas	1
RF_07	Could	Como administrador quiero identificar a los tutores con mayor demanda para planificar la asignación de recursos.	Backend: Consultas y agregaciones sobre historial de tutorías	3
			Frontend: Vista de ranking/reporte de demanda por tutor	2
			Integración y pruebas	1
RF_08	Could	Como administrador quiero almacenar los resultados de evaluaciones y cuestionarios para generar reportes de desempeño.	BD: Modelo de almacenamiento de resultados de cuestionarios	2
			Backend: Lógica de consolidación y generación de reportes	3

			Frontend: Vista de reportes con filtros por periodo/materia/estudiante	3
			Integración y pruebas	2
RF_09	Should	como estudiante quiero poder cancelar o reprogramar una tutoría con anticipación Para reorganizar mi horario sin afectar al tutor ni perder mi cupo.	Backend: Validación de anticipación mínima, liberación de cupo y notificación al tutor	3
			Frontend: Opción de cancelar/reprogramar desde el historial de tutorías	3
			Integración y pruebas	2
RF_10	Should	Como tutor, quiero visualizar el historial de tutorías de cada estudiante antes de atenderlo, para conocer qué temas ya se trabajaron y así preparar mejor la sesión.	Diseño y frontend para cancelar/reprogramar	3
			Backend: Validación, disponibilidad y notificaciones	4
			Integración y pruebas	2
RF_11	Must	Como administrador, quiero poder gestionar los usuarios del sistema (crear, modificar o desactivar cuentas de estudiantes y tutores), para mantener actualizada y organizada la información institucional.	Diseño y frontend del panel de administración.	3
			Backend: CRUD, roles, permisos y desactivación.	5
			Integración y pruebas.	2
RF_12	Could	Como estudiante, quiero consultar los resultados de mis evaluaciones y comentarios del tutor después de cada sesión, para identificar mis fortalezas	Diseño y frontend de resultados con indicadores.	3
			Backend: Modelo de datos, endpoints y feedback.	4

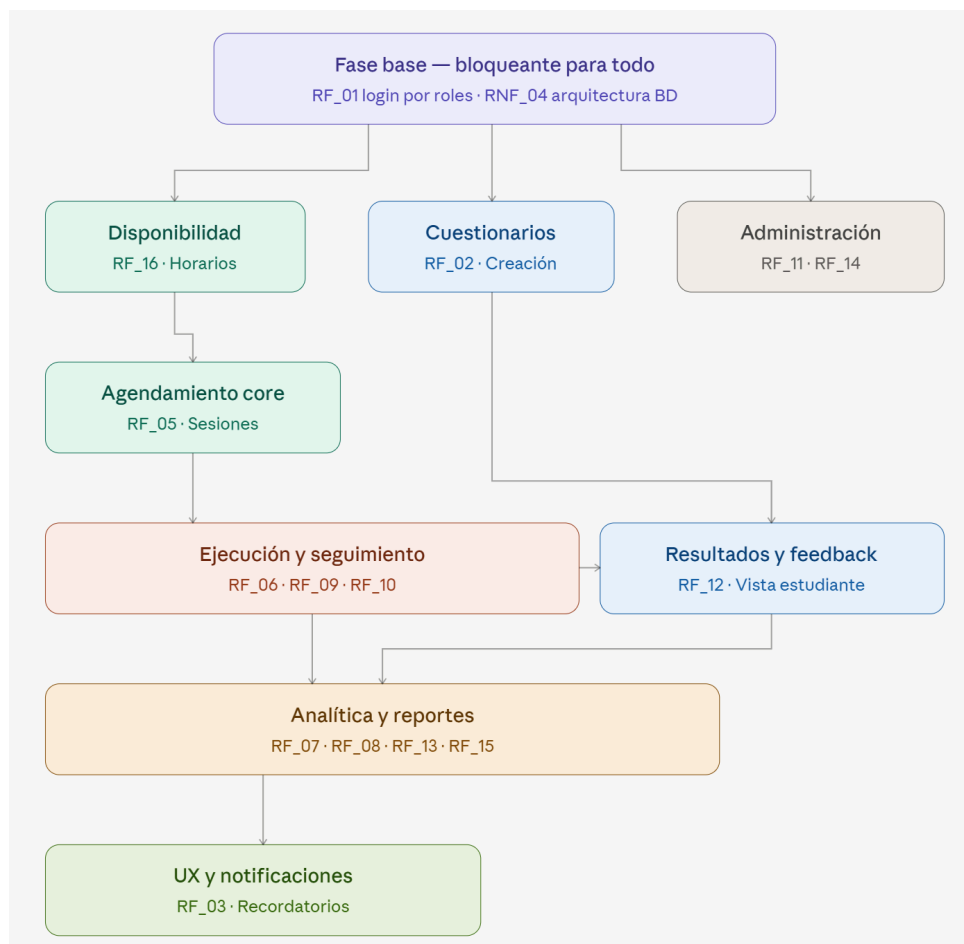
		y debilidades y mejorar mi rendimiento académico.	Integración y pruebas	2
RF_13	MUST	Como administrador quiero poder revisar las ausencias de los estudiantes para evaluar la participación activa de los mismos en el sistema.	Creación de tablas de datos.	3
			Backend para lógica de consulta.	2
			Fronted para la vista del administrador.	3
			Integración y pruebas.	1
RF_14	SHOULD	Como usuario quiero tener una forma de ver toda la información de los distintos usuarios.	Creación y asignación de roles.	1
			Backend para trabajar los datos.	2
			Fronted para la vista de manera detallada de los usuarios.	3
			Integración y pruebas.	1
RF_15	MUST	Como estudiante quiero tener la posibilidad de ver mis notas y avances en las tutorías.	Creación de tablas de datos	2
			Backend para la búsqueda en base de datos.	3
			Fronted para la visualización de notas y avance en las tutorías (Historial).	3
			Integración y pruebas.	2
RF_16	MUST	Como tutor quiero gestionar mi disponibilidad horaria en el sistema para que los estudiantes solo puedan agendar en mis bloques de tiempo libres	Creación de la tabla de datos de disponibilidad.	1
			Backend para la solicitud del tutor.	2

			Fronted para la visualización de horarios manipulables.	2
			Integración y pruebas.	1

3.1 Resultados del Poker Planning y Cronograma de Dependencias

Con base en la regla de estimación máxima, el equipo realizó una sesión de Poker Planning (basada en días de desarrollo) donde ninguna tarea supera los 3 días. A partir de estos puntajes, se construyó el árbol de dependencias lógicas: lo que debe construirse primero para desbloquear las siguientes funcionalidades.

Árbol de dependencias lógicas



3.2 Organización del Tablero (jira)

<https://unal-team-tsrh38b0.atlassian.net/jira/software/projects/SCRUM/boards/1/backlog>